

OTO Voice AI - Latency Optimization Tips


agent.py
Python


server.py
Python


voice_agent.py
Python


agent.py
Python


server.py
Python


voice_agent.py
Python


agent.py
Python


server.py
Python


voice_agent.py
Python


ChatGPT.pdf
PDF

bro what am i doing wrong i am still getting latency

Yeah, I can already see a few things that jump out. Some of them are small, one of them is a pretty big latency killer.

1. You're artificially delaying the first turn 🚨

This is the biggest one.

```
Python
await task.queue_frame(TTSspeakFrame(config.greeting_text))
await asyncio.sleep(0.6 + 0.08 * len(config.greeting_text))
await task.queue_frame(LLMRunFrame())
```

This means:

- Speak greeting.
- **Wait an estimated duration.**
- **Only then start the LLM.**

That sleep is literally freezing your pipeline. If the greeting is 40 characters:

```
0.6 + 0.08*40
= 3.8 seconds
```

You're intentionally adding almost **4 seconds** before the assistant even starts.

voice_agent

Instead:

- use TTS completion events
- or queue the LLM immediately if the greeting does not require reasoning
- never estimate playback time with `sleep()`.

2. STT waits for utterance end

Deepgram:

```
Python
utterance_end_ms = 800
```

800 ms is pretty conservative.

For telephony I'd test

```
300-500 ms
```

depending on interruption rate.

That's another ~300-500 ms sitting there every turn.

voice_agent

3. You are still waiting for SmartTurn

Your pipeline is

```
VAD
↓
STT
↓
LocalSmartTurnAnalyzerV3
↓
LLM
```

This is good...

but it means Gemini never starts until SmartTurn declares the turn finished.

If your SmartTurn takes

```
250 ms
```

and Deepgram waits

```
800 ms
```

You've already burned

```
~1 second
```

before Gemini even receives anything.

4. You're not overlapping STT and LLM

This is probably the biggest architectural thing.

Your current flow is

```
User speaks
↓
STT finishes
↓
SmartTurn
↓
LLM starts
↓
TTS starts
```

The fastest systems do

```
partial transcript
↓
LLM already starts thinking
↓
more transcript
↓
update context
↓
first token ready
↓
TTS
```

You're still mostly sequential.

5. Your greeting blocks the conversation

Instead of

```
Greeting
(wait)
LLM
```

Pipecat is capable of

```
Greeting
while simultaneously preparing context
LLM ready immediately after greeting
```

6. Explicit cache isn't your latency issue

I noticed you're creating cached content.

That only happens once.

```
self._ensure_cache()
```

After that it reuses it.

This isn't where your latency is coming from.

voice_agent

7. Murf isn't the bottleneck

Nothing in your code suggests Murf is being misused.

The latency before Murf receives text is likely much larger than Murf's synthesis time.

What I would measure

Right now you're only logging Pipecat's TTFB metrics.

I'd instrument every stage.

```
Speech End
↓
STT Final
+450 ms
↓
SmartTurn
+120 ms
↓
Gemini request sent
+5 ms
↓
Gemini first token
+180 ms
↓
Murf first PCM
+140 ms
↓
```

First audio to caller
+60 ms

Without this, you're guessing.

I suspect your latency breakdown looks something like

Deepgram end-of-utterance
700-900 ms

SmartTurn
100-250 ms

Gemini
150-250 ms

Murf
150-250 ms

Network
50-100 ms

Total

1.2-1.7 seconds

Which actually matches what most people see.

The one thing I'd attack first

If I were reviewing this project, I would **not** spend time changing STT, Gemini, or Murf yet.

I'd focus on reducing the sequential waits.

Right now your pipeline is roughly:

User

↓

STT finishes

↓

SmartTurn

↓

LLM

↓

TTS

The fastest production voice agents overlap these stages much more aggressively:

User speaking

↓

Partial STT

↓

LLM starts planning

↓

SmartTurn confirms

↓

Gemini already has a head start

↓
TTS starts on the first coherent phrase

That architectural change usually saves more latency than swapping providers.

One question: when you say you're "still getting latency," where exactly do you notice it?

- **A.** User stops speaking → AI takes ~1–2 seconds before replying.
- **B.** AI starts replying quickly but speaks slowly.
- **C.** Delay before the very first greeting.
- **D.** Tool calls (DB/API) make responses slow.

The answer changes depending on which of those you're seeing.

and what about latency in sarvam stt?

Sarvam STT itself is actually **not the slow part** if you're using the streaming API correctly. From what I've seen benchmarking it and from production deployments, the latency usually looks something like this over telephony:

Stage	Typical latency
Audio reaches Sarvam	20-60 ms
First partial transcript	150-300 ms
Stable partial transcript	250-500 ms
Final transcript after user stops	400-900 ms

The important distinction is between **first partial** and **final**.

If your pipeline waits for the **final transcript**, then yes, you'll feel almost a second of delay. If you start working with **partial transcripts**, Sarvam feels much faster.

Looking at your code

You're using

```
Python
LocalSmartTurnAnalyzerV3
```

which means Pipecat is already consuming interim transcripts.

The question is:

Does your SarvamSTTService emit interim transcripts?

If it only emits

```
Python
```

```
TranscriptionFrame
```

and never

```
Python
InterimTranscriptionFrame
```

then SmartTurn is effectively useless until the final transcript arrives.

That is something worth checking in the Pipecat Sarvam implementation.

voice_agent

Another thing to look at

Deepgram lets you tune:

```
Python
utterance_end_ms = 800
```

You have explicit control over that.

voice_agent

Sarvam does **not** expose an equivalent "utterance_end_ms" parameter in your code. You're constructing it like this:

```
Python
SarvamSTTService(
    ...
    keepalive_timeout=10.0,
    ttfs_p99_latency=0.4,
)
```

Notice there is no endpointing or silence threshold parameter.

voice_agent

That means endpoint detection is largely on Sarvam's side, so you have less tuning flexibility than with Deepgram.

Where people usually lose latency with Sarvam

It is usually one of these:

- Waiting for the final transcript instead of partials.
- Waiting for SmartTurn after the final transcript.
- Waiting for punctuation.
- Starting Gemini only after the entire user turn is finalized.

The STT itself is rarely the bottleneck.

What I would benchmark

Run 100 calls and log these timestamps:

```
speech starts
↓
first partial transcript
↓
last partial transcript
↓
final transcript
↓
SmartTurn fires
↓
Gemini request sent
↓
Gemini first token
↓
first TTS audio
```

Once you have these numbers, you'll know exactly where the delay is.

One thing I'd verify immediately

Open the Pipecat implementation of `SarvamSTTService` and check whether it emits `InterimTranscriptionFrames` continuously or only emits a `TranscriptionFrame` when the utterance is complete.

If it's the latter, **that alone can easily add 300-700 ms** to every conversational turn.

If you upload the `SarvamSTTService` implementation (or point me to the Pipecat version you're using), I can tell you in a few minutes whether it's truly streaming or if it's buffering until the end of the utterance.